

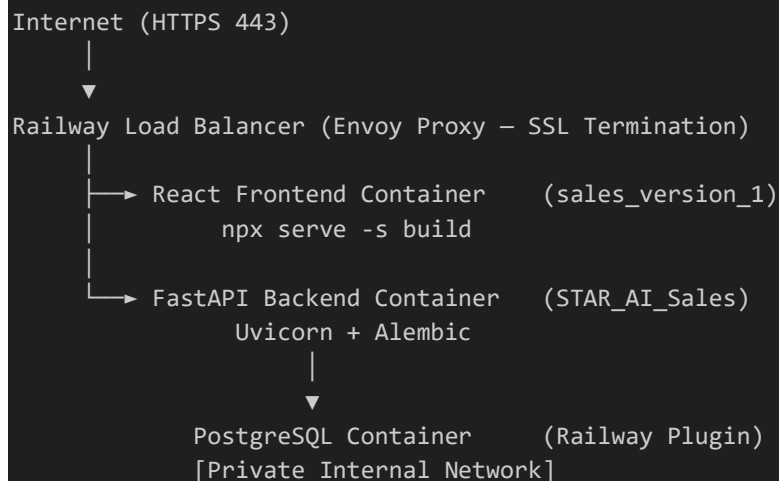
STAR AI Sales — Railway Deployment Guide

Complete step-by-step process for deploying the backend (FastAPI), database (PostgreSQL), and frontend (React) to Railway.app

Table of Contents

1. Architecture Overview
2. Phase 0 — Pre-Deployment: Push to GitHub
3. Phase 1 — Database Setup on Railway
4. Phase 2 — Backend Deployment (FastAPI)
5. Phase 3 — Frontend Deployment (React)
6. Phase 4 — Connect Frontend ↔ Backend (CORS)
7. Phase 5 — Verify Everything is Working
8. Future Schema Changes with Alembic
9. Environment Variables Quick Reference
10. Troubleshooting

Architecture Overview



Key facts:

- All 3 services run as **separate containers** in one Railway Project
- PostgreSQL is on Railway's **private internal network** — not exposed to the public internet
- DATABASE_URL is **auto-injected** by Railway into the backend container
- The backend runs `alembic upgrade head` **automatically on every startup** before serving traffic

Phase 0 — Pre-Deployment: Push Code to GitHub

Do this **before** opening Railway. Railway pulls code directly from GitHub.

Step 1 — Verify your .gitignore

Ensure these files are **never committed**:

```
gitignore
# STAR_AI_Sales/.gitignore
app/assets/.env
*.pem
star_ai11_backup.sql
star_ai1_dump.sql
# sales_version_1/.gitignore
/node_modules
.env
```

Step 2 — Confirm all Railway config files exist

Check that these files are present in STAR_AI_Sales/:

File	Purpose
railway.toml	Tells Railway how to build and start the backend
nixpacks.toml	Pins Python 3.11 and system dependencies
start.sh	Runs Alembic migrations then starts Uvicorn
alembic.ini	Alembic configuration (no hardcoded DB URL)
alembic/env.py	Reads DATABASE_URL dynamically from environment

And in sales_version_1/:

File	Purpose
railway.toml	Tells Railway to run <code>npm run build</code> and <code>serve -s build</code>
.env.production	Placeholder env vars (overridden by Railway dashboard)

Step 3 — Push to GitHub

```
bash
# Backend
cd STAR_AI_Sales
git add .
git commit -m "feat: Railway deployment configuration"
git push origin main
# Frontend
cd ../sales_version_1
git add .
git commit -m "feat: Railway frontend configuration"
```

```
git push origin main
```

☒ Code is now on GitHub. You are ready to deploy.

Phase 1 — Database Setup on Railway

The database **must be created first** so the backend can connect to it on first boot.

Step 1 — Create a New Railway Project

1. Go to railway.app and log in
2. Click **New Project**
3. Select **Provision PostgreSQL**
4. Railway creates a fresh, empty PostgreSQL container

Step 2 — Choose Your Database Setup Method

You have **two options**:

Ⓐ Option A — Let Alembic Create Everything (Fresh Start)

☒ **Use this if you are starting fresh with no existing data to migrate.**

You do nothing here. Leave the database empty.

When you deploy the backend in Phase 2, `start.sh` automatically runs:

```
bash
```

```
alembic upgrade head
```

Alembic executes all **27 migration files** in order and builds the entire schema:

20260806_init_sales_schema	→ sales schema, contacts, leads, leaders tables
20260806_sales_dashboard_mv	→ materialized views for dashboard
20260811_sales_notifications	→ notifications tables
20260811_sales_str_pks	→ primary key changes
20260812_sales_notif_mv	→ notification materialized views
20260813_change_product_register...	→ numeric type changes
20260813_shift_pv_ec_to_product	→ product table changes
20260818_leader_reassignment_history	→ leader history table
20260818_update_application_tables	→ application table updates
20260818_update_sales_notifications	→ notification MV update
20260819_add_lost_reason...	→ product register columns
20260819_add_proposal_columns	→ proposal columns
20260820_add_lead_id_fk_to_contacts	→ foreign key constraint
20260820_update_sales_notif_fk	→ FK cascade update
20260821_add_created_by_to_contacts	→ contacts column
20260821_remove_lead_id_from_contacts	→ contacts cleanup
20260824_add_updated_at_to_lead...	→ timestamp columns
20260828_add_meeting_action_remarks	→ remarks column
20260828_add_overdue_reason	→ overdue reason column
20260828_add_role_to_leaders	→ leader role column
20260828_create_proposal_sent	→ proposal sent table
20260831_add_product_owner_id	→ product owner column

20260902_schedule_pg_cron_mv_refresh → pg_cron scheduled refresh

20260903_create_enhanced_login_history → login history table

20260910_add_response_payload... → mail events column

20260911_add_password_reset_otp_table → OTP table

20260916_add_otp_and_weekly_report_email → email env table

After this, the database will have all tables, indexes, schemas, and materialized views — **ready to use with zero manual steps.**

⚠ The database will be empty (no leaders, no leads, no sales data). You will need to create your leaders/users through the application after deployment.

ⓑ Option B — Restore Existing Data from SQL Backup

☒ **Use this if you have existing sales/lead data you need to preserve.**

1. Click on the **PostgreSQL service** in Railway
2. Go to the **Connect** tab
3. Copy the **Public Connection URL**, which looks like:

<postgresql://postgres:PASSWORD@monorail.proxy.rlwy.net:PORT/railway>

4. On your local machine, run:

```
bash
```

```
psql "postgresql://postgres:PASSWORD@monorail.proxy.rlwy.net:PORT/railway" \  
-f star_ai11_backup.sql
```

Or use **pgAdmin / DBeaver** → connect with the credentials → Run SQL file.

5. All your existing tables, data, and records are now restored.

☒ When the backend deploys, Alembic will detect all tables already exist via the `alembic_version` table and **skip re-creating them** — your data is safe.

Phase 2 — Backend Deployment (FastAPI)

Step 1 — Add Backend Service

1. In your Railway Project → click + **New** → **GitHub Repo**
2. Select your STAR_AI_Sales repository
3. Railway detects `railway.toml` and starts building

⚠ The service may crash on first boot because environment variables are not set yet. This is normal.

Step 2 — Link the PostgreSQL Database

1. Click **Backend service** → **Variables** tab
2. Click **Add Reference** → select the PostgreSQL plugin
3. Railway auto-injects `DATABASE_URL` in this format:

<postgresql+asyncpg://postgres:PASSWORD@postgres.railway.internal:5432/railway>

💡 Your `alembic/env.py` reads `settings.asyncpg_url` which is derived from `DATABASE_URL`. This is already configured correctly — no changes needed.

Step 3 — Mount a Persistent Volume

⚠ Critical. Without a volume, all uploaded files are permanently lost on every container restart.

1. Click Backend service → **Settings** tab
2. Scroll to **Volumes** → click **Add Volume**
3. Set the **Mount Path** to: /data
4. Save

Step 4 — Set All Environment Variables

Click Backend service → **Variables** tab → add each one:

Variable	Value	Required?
DATABASE_URL	Auto-injected by PostgreSQL plugin	✓ Auto
JWT_SECRET_KEY	Run openssl rand -hex 32 locally and paste the result	✓ Required
SQL_DB_PATH	/data	✓ Required
UPLOAD_BASE_DIR	/data/uploads	✓ Required
ENABLE_IP_RESTRICTION	false	✓ Recommended
IS_PRODUCTION	true	✓ Required
FRONTEND_URL	(Add after Phase 3 — leave blank for now)	✓ Required
SUPERSET_HOST	http://your-superset.com	● Optional
SUPERSET_USERNAME	Superset admin username	● Optional
SUPERSET_PASSWORD	Superset admin password	● Optional
SEAWEEDFS_FILER_URL	Your SeaweedFS URL (skip if not using file storage)	● Optional

Step 5 — Watch Alembic Run in the Logs

After saving variables, the backend redeploys. Click **Logs** to watch:

```
===== [1/2] Running Alembic Migrations =====
INFO [alembic.runtime.migration] Context impl PostgreSQLImpl.
INFO [alembic.runtime.migration] Will assume transactional DDL.
INFO [alembic.runtime.migration] Running upgrade -> 20260806_init_sales_schema
INFO [alembic.runtime.migration] Running upgrade -> 20260806_sales_dashboard_mv
INFO [alembic.runtime.migration] Running upgrade -> 20260811_sales_notifications
```

```
... (all 27 migrations run automatically)
INFO [alembic.runtime.migration] Running upgrade -> 20260916_add_otp_and_weekly...
===== [2/2] Starting Uvicorn Server =====
INFO:      Started server process
INFO:      Uvicorn running on http://0.0.0.0:PORT (Press CTRL+C to quit)
☒ If all migrations complete and Uvicorn starts — the backend is live!
```

Step 6 — Generate Backend Domain

1. Backend service → **Settings** → **Networking**
2. Click **Generate Domain**
3. Copy the URL:

<https://star-ai-backend.up.railway.app>

Step 7 — Verify the Backend Health Check

Open in your browser:

<https://star-ai-backend.up.railway.app/health>

Expected response:

json

```
{"status": "ok"}
```

☒ Backend is running correctly.

Phase 3 — Frontend Deployment (React)

Step 1 — Add Frontend Service

1. Railway Project → + **New** → **GitHub Repo**
2. Select `sales_version_1` repository
3. If the frontend is a sub-folder of your repo:
 - a. Click the service → **Settings** → **Root Directory**
 - b. Set to `sales_version_1`

Step 2 — Set Environment Variables

☒ No code changes needed. The React app already reads `REACT_APP_BACKEND_URL` from the environment.
Click Frontend service → **Variables** tab → add:

Variable	Value
<code>REACT_APP_BACKEND_URL</code>	https://star-ai-backend.up.railway.app ← from Phase 2 Step 6
<code>REACT_APP_BUCKET_BASE_URL</code>	Your SeaweedFS/R2 public URL (optional)

💡 Railway injects these during `npm run build` so they get baked into the production React bundle automatically.

Step 3 — Verify the Build

Railway automatically runs:

```
bash
npm run build          # Creates the optimized /build folder
npx serve -s build -l $PORT # Serves the SPA
```

The `-s` flag ensures React Router paths like `/dashboard` and `/leads` work without returning 404 errors on refresh.

Step 4 — Generate Frontend Domain

1. Frontend service → **Settings** → **Networking**
2. Click **Generate Domain**
3. Copy the URL:

<https://star-ai-frontend.up.railway.app>

Phase 4 — Connect Frontend ↔ Backend (CORS)

Without this step, the browser will block all API calls with a **CORS error** even if both services are running correctly.

Step 1 — Add Frontend URL to Backend Variables

1. Go to **Backend service** → **Variables** tab
2. Add:

Variable	Value
FRONTEND_URL	https://star-ai-frontend.up.railway.app

Step 2 — Backend Auto-Restarts

Saving a variable triggers an automatic redeploy. Once the backend restarts with the new CORS config, your full-stack application is fully connected and live.

Phase 5 — Verify Everything is Working

Run through this checklist:

Backend health check: <https://star-ai-backend.up.railway.app/health> returns `{"status": "ok"}`

Frontend loads: <https://star-ai-frontend.up.railway.app> shows the login page

Login works: Credentials accepted → redirected to dashboard (cross-origin cookies working)

Data visible: Dashboard shows data (Alembic ran successfully + data is present)

No 404 on refresh: Refreshing `/dashboard` still loads correctly (SPA mode working)

File uploads work: Upload a document (only if SeaweedFS is configured)

Future Schema Changes with Alembic

Whenever you add a new table or column to your models, follow this workflow:

Step 1 — Update your SQLAlchemy Model

Edit the relevant model file in `app/models/`.

Step 2 — Generate a New Migration File

On your **local machine**, run:

```
bash
cd STAR_AI_Sales
alembic revision --autogenerate -m "describe_your_change_here"
```

This creates a new file in `alembic/versions/` like:

```
alembic/versions/20260923_describe_your_change_here.py
```

Step 3 — Review the Generated Migration

Open the file and verify the `upgrade()` and `downgrade()` functions look correct.

Step 4 — Test Locally (Optional but Recommended)

```
bash
alembic upgrade head
```

Step 5 — Commit and Push

```
bash
git add alembic/versions/20260923_describe_your_change_here.py
git commit -m "migration: describe your change here"
git push origin main
```

Step 6 — Railway Handles Everything Automatically

Railway detects the new commit → rebuilds the container → runs `start.sh` → Alembic applies the new migration → server starts.

☒ **Zero manual steps on Railway.** Every future schema change is handled automatically on deploy.

Environment Variables Quick Reference

Backend (STAR_AI_Sales)

Variable	Example Value	How to Get
DATABASE_URL	postgres://postgres:...@postgres.railway.internal:5432/railway	Railway Auto
JWT_SECRET_KEY	a3f8b2c1d4e5... (64 hex chars)	openssl rand

		-hex 32
		Type manually
SQL_DB_PATH	/data	
		Type manually
UPLOAD_BASE_DIR	/data/uploads	
		Type manually
ENABLE_IP_RESTRICTION	false	
		Type manually
IS_PRODUCTION	true	
		Type manually
FRONTEND_URL	https://star-ai-frontend.up.railway.app	After Phase 3
SEAWEEDFS_FILER_URL	http://your-vps-ip:8888	Optional

Frontend (sales_version_1)

Variable	Example Value	How to Get
REACT_APP_BACKEND_URL	https://star-ai-backend.up.railway.app	After Phase 2
REACT_APP_BUCKET_BASE_URL	https://pub-xxx.r2.dev	Optional

 Troubleshooting

Backend crashes on first deploy

Check: All environment variables are set, especially DATABASE_URL and JWT_SECRET_KEY.

Alembic fails with "relation already exists"

Cause: The SQL backup was restored but has no alembic_version table.

Fix: Connect to the Railway DB and run:

```
sql
```

```
CREATE TABLE IF NOT EXISTS alembic_version (  
  version_num VARCHAR(128) NOT NULL PRIMARY KEY  
);  
INSERT INTO alembic_version (version_num)  
VALUES ('20260916_add_otp_and_weekly_report_email_from_to_emailenv');
```

This tells Alembic all 27 migrations are already applied.

Login works but I get logged out immediately

Cause: Cookies require Secure=True for SameSite=None.

Fix: Ensure IS_PRODUCTION=true is set in backend variables.

Frontend shows a blank page or 404 on route refresh

Cause: SPA mode (-s flag) is missing in the serve command.

Fix: Verify sales_version_1/railway.toml:

```
toml  
[deploy]  
startCommand = "npx serve -s build -l $PORT"
```

API calls fail with CORS error

Cause: FRONTEND_URL not set or value doesn't exactly match the frontend domain.

Fix: Backend service → Variables → Set FRONTEND_URL to the exact frontend Railway domain (include <https://>).

Alembic fails with "could not connect to server"

Cause: DATABASE_URL not linked to the backend service.

Fix: Backend service → Variables → Add Reference → Select the PostgreSQL plugin.

STAR AI Sales — Railway Deployment Guide — September 2026

-
-
-
-
-